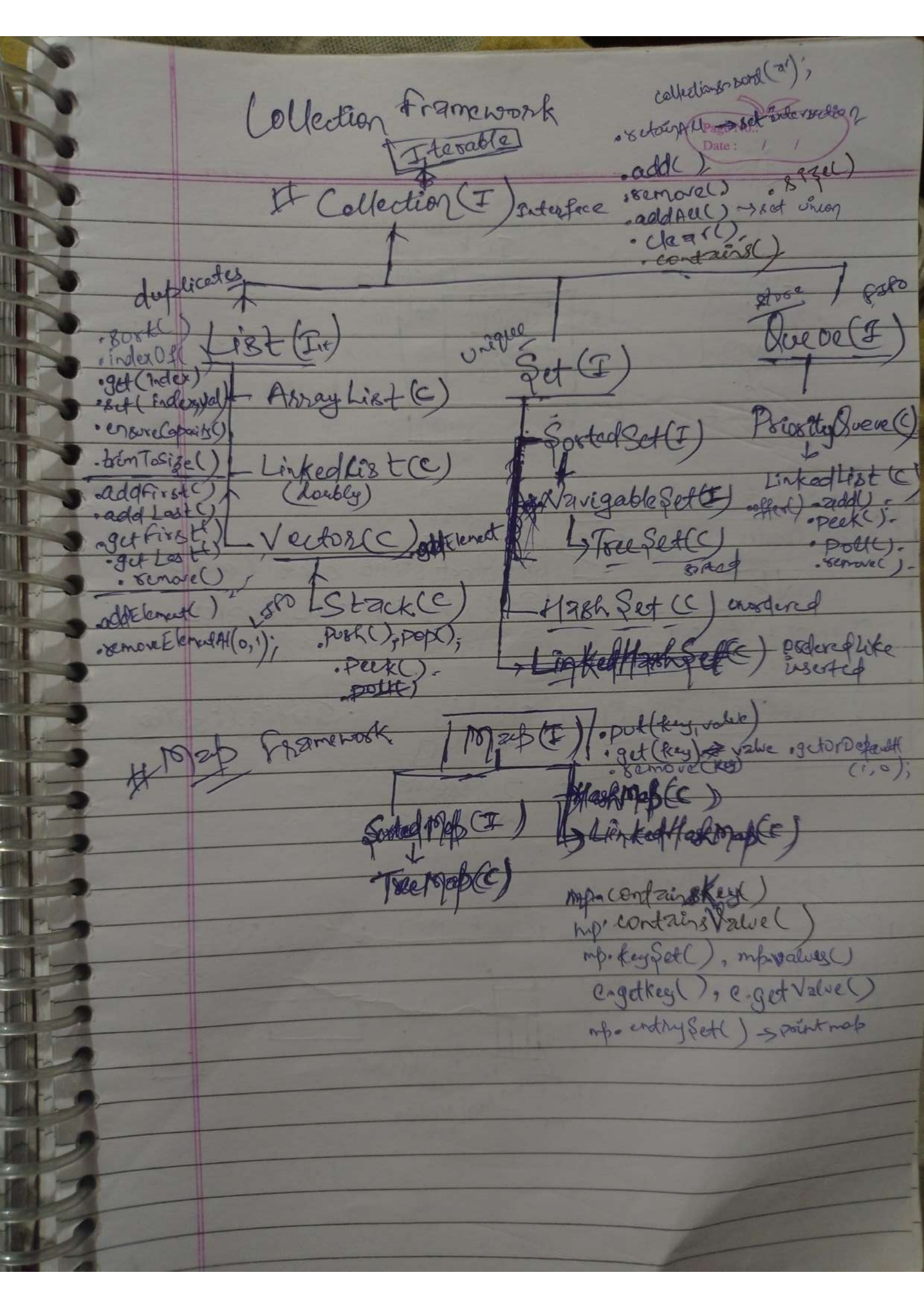
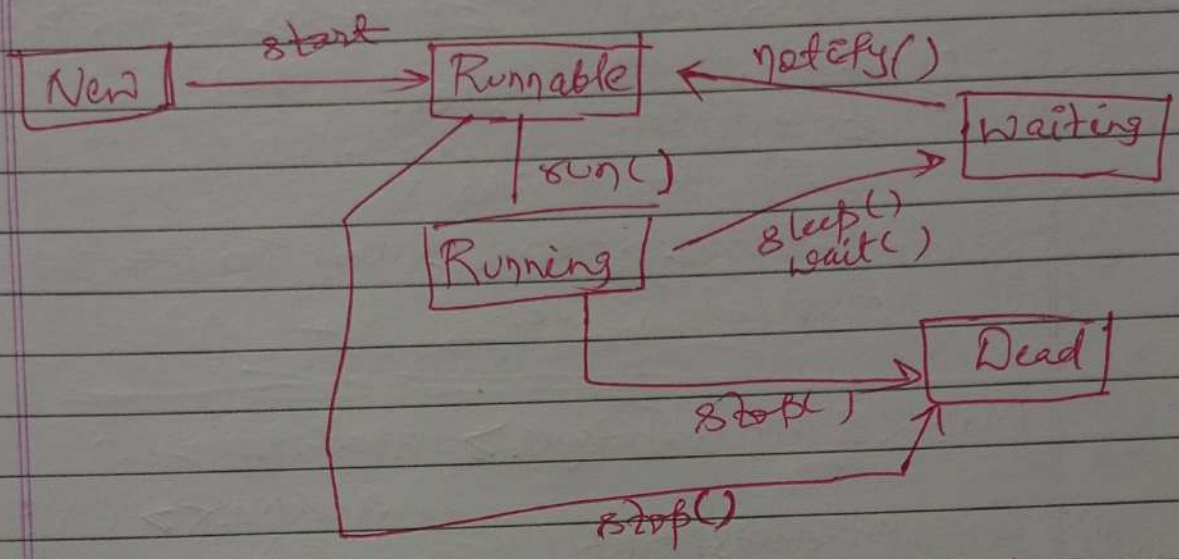
Collection framework



Thread states.



- Collection API → concept → Map.
- Collection → ~~method~~ (interface)
- Collections → class (methods)

Iterator

Collection

- List
  - ArrayList
  - LinkedList
- Queue
  - DeQueue
- Set
  - HashSet, TreeSet
  - LinkedHashSet

```
Collection<Integer> nums = new ArrayList<Integer>
nums.add();
```
• can't get index

* to initialize value → Collections.nCopies(size, value)

```
List<Integer> nums = new ArrayList<Integer>();
nums.add();
nums.get(2);
nums.indexOf(2);
Collections.sort(nums);
nums.contains();
.set(index, val);
```

• Can't get value by index
• contains duplicate element

Collection class
↓
Set class
↓
TreeSet class

Collection

Set<Integer> nums = new HashSet<Integer>();
nums.add();
- unique values
- unordered.
- can't get by index

Set<Integer> nums = new TreeSet<Integers>();
- sorted values.

Map<String, Integer> students = new HashMap<>();
students.put("Sam", 3);
- keys can't be duplicate

students.get("Sam");
students.keySet();
for(String name : students.keySet())
{ sout(key + " " + students.get(key));}

- .isEmpty()
- .remove()
- .getOrDefault(1,0)
- .containsKey()
- .containsValue()

Map<String, Integer> stud = new HashTable<>();
- Synchronized externally.

- It's an interface.

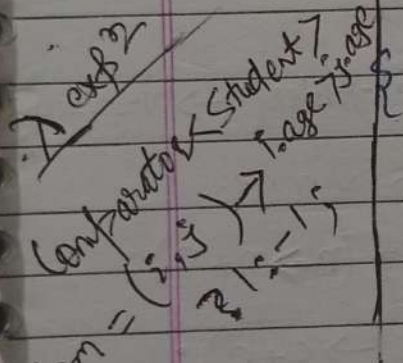
Comparator<Integers> com = new Comparator<Integer>()
{ public int compare(Integer i, Integer j)
{ if(i%10 > j%10)
return 1;
else return -1;
}
};

List<Integers> nums = new ArrayList<>();
nums.add(43), 31, 72, 29.
Collections.sort(nums, com);
print(nums); 31, 72, 43, 29

Page No.:
Date: / /

Q Create List of String & sort according to length of String.

```
Class Student{ int age;
                String name;
public Student (int age, String name){
        this.age = age ; this.name = name;}
public String toString(){
    return "Student [age="+age +",name="+
                name +"]" ;}
}
main){
    Comparator<Student> com = new ~~ComparatorStude~~
                Comparator<Student>()
public int Compare(Student i, Student j)
    { if(i.age > j.age) return 1;
      else return -1;
    }
};
List<Student> studs = new ArrayList<>();
studs.add(new Student (21, "Navin"));
          (12, "Sam"));
          (18, "Pankaj"));
Collections.sort(studs, com);
for(Student s: studs)
        sout(s);
```



Page No.:
Date: / /

* List<Integer> nums = new ArrayList<>();
= Arrays.asList(4,5,12);

→ for(int i=0; i<nums.size(); i++)
{ print(nums.get(i); }

→ for(int n : nums)
{ print(n); }

Stream API → nums.forEach(n → sout(n));

java.util.stream.Stream

Stream<Integer> s1 = nums.stream(); ← all value on that list
(original copy)

can't use twice

s1.forEach(n → sout(n));

Stream<Integer> s2 = s1.filter(n → n%2==0);
s2.forEach(n → sout(n)); → even no.
Stream<Integer> s3 = s2.map(n → n*2);
int result = s3.reduce(0,(c,e) → c+e);
print(result)

int result = nums.stream() ← it gives stream of values
.filter(n → n%2==0)
.map(n → n*2)
.reduce(0,(c,e) → c+e);
print(result);

initial, carry, element

Stream API

Page No.:
Date : / /

```
Stream<Integer> sortedValues = nums.stream()
                    .filter(n -> n%2==0)
                    .sorted();
sortedvalues.forEach(n -> sout(n));
```

We can use nums.parallelStream()
to filter values

- Multi-Threading
- Exception Handling } Core Java
- File Handling
- Database Connectivity } JDBC
- Dependency Injection
- Servlet
- Hibernate (JPA)
- Spring Security
- Microservices (begin)
- Docker

Page No.:
Date: / /

- Map ordered

```
Set<Character> distincts = new LinkedHashSet<>();
in loop
    distincts.add(str.charAt(i));

adv. loop
    uniqueString += String.valueOf(c);
```

```
import java.util.Arrays;
int arr[] = new[]{2,3,1,5,6,7,8,7,5,6};
```

→ ~~Arrays.sort(arr);~~
→ Arrays.binarySearch(arr, 120);
→ String s = Arrays.toString(arr);
→ Arrays.equals(arr, arr1);
→ Arrays.fill();

a=8, b=9

→ Integer.toBinaryString(a) → Binary string
→ Integer.toBinaryString(~~a&b~~ a|b);

→ Arrays.toString(arr) → prints whole array at a time

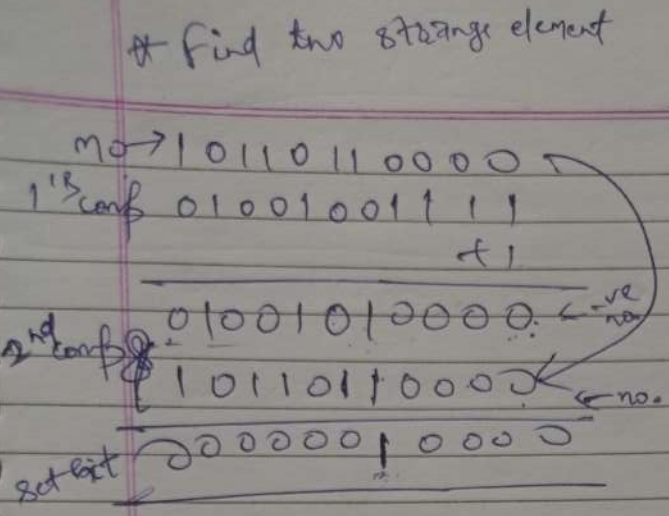
Integer.toBinaryString(-a >>> 2) unsigned no. shift

Integer.toBinaryString(-a << 2); signed no. shift

" (-a); 11111111 11111111 11111111 11111000

signed
unsigned

* Find two strange element



Page No.:
Date: / /

2nd complement

x & (-x)

x & (~x+1)

111
1 1 1 1 1 1 0

* ArrayList a = new ArrayList ()

- a.add()
- remove()
- clear()
- contains()
- size()

{ - contains diff types of data types ele. int, float, string, obj.